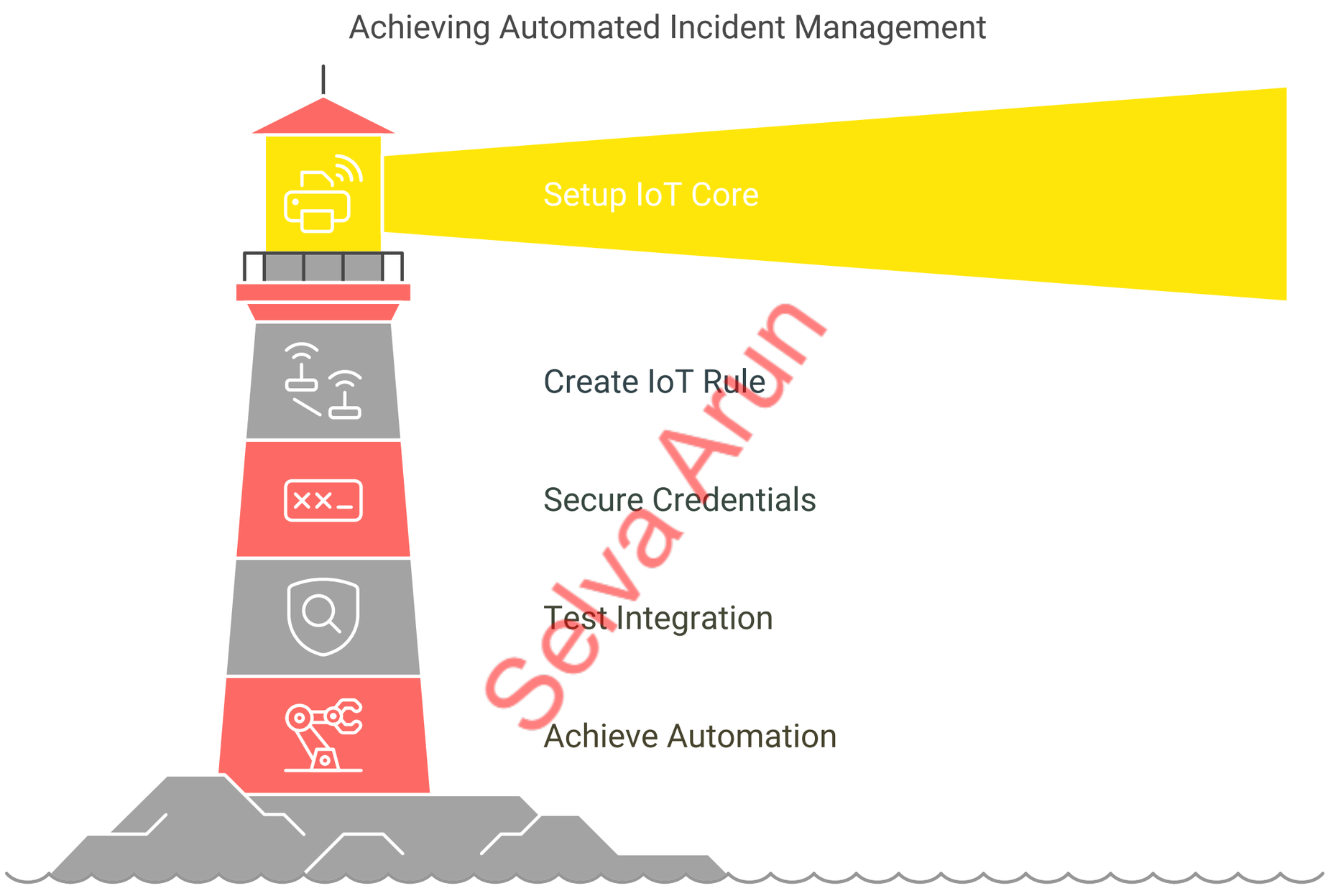
Integrating AWS IoT Core with ServiceNow for Automated Incident Management

In this document, we explore the integration of AWS IoT Core with ServiceNow to create an automated incident management solution. This integration aims to enhance operational efficiency by enabling proactive maintenance and minimizing downtime through early detection of potential issues. By leveraging IoT sensors and ServiceNow's incident management capabilities, organizations can transform their approach to IT issues, ensuring a more responsive and streamlined process.



Video Tutorial Available!

Check out my YouTube channel for the full video tutorial: Part 1: IoT and ITSM Integration: AWS IoT Core + ServiceNow for Incident Automation

Note: This is Part 1 of my integration series. I will be posting Part 2 soon, where I'll demonstrate building a Flow Designer solution for automating this process end-to-end.

Introduction

In today's rapidly evolving IT landscape, organizations are constantly seeking ways to minimize downtime and respond to issues before they impact business operations. By combining the power of AWS IoT Core with ServiceNow's incident management capabilities, we can create a proactive system that automatically detects and responds to potential problems. This guide walks you through integrating these powerful platforms to create an automated incident management solution that transforms how your organization handles IT issues.

Why IoT is Important for Incident Management

The Internet of Things (IoT) is revolutionizing how businesses operate by enabling:

- Proactive maintenance instead of reactive troubleshooting
- Minimized downtime through early detection of potential issues
- Enhanced operational efficiency with automated workflows

In our integration, IoT sensors continuously monitor environmental conditions such as temperature, humidity, or power usage. When these sensors detect values exceeding predefined thresholds, they trigger the automatic creation of incidents in ServiceNow, alerting IT teams for immediate response and resolution.

Step-by-Step Integration Guide

Step 1: Set Up AWS IoT Core

1.1 Create an IoT Thing

1. Log in to the AWS IoT Core Console: [AWS IoT Console](https://us-east-2.console.aws.amazon.com/iot/home)
2. Navigate to Manage > Things
3. Click Create things and select Create a single thing
4. Enter a name for your IoT device (e.g., TemperatureSensor1)
5. Add optional attributes:
    - type: sensor
    - location: datacenter
6. Click Next

1.2 Generate Device Certificates

1. Select Auto-generate a new certificate
2. Download the following files:
    - Device certificate
    - Public key
    - Private key
    - Amazon Root CA certificate
3. Store these files securely as they're essential for device authentication

1.3 Attach a Policy to the Certificate

1. Navigate to Secure > Certificates
2. Find the certificate and select Attach policy
3. Create a new policy with the following JSON:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "iot:Connect",
        "iot:Publish",
        "iot:Subscribe",
        "iot:Receive"
      ],
      "Resource": "*"
    }
  ]
}
```

4. Attach the policy to the certificate

Step 2: Create an IoT Rule

2.1 Define the Rule

1. Go to Message routing > Rules
2. Click Create rule
3. Enter TemperatureAlertRule as the rule name

2.2 Configure the SQL Query

Use this query to filter messages from the MQTT topic:

```
SELECT * FROM 'iot/sensors' WHERE sensors = 'temperature'
```

Update 'iot/sensors' based on your MQTT topic structure.

2.3 Add a Rule Action

1. Click Add action > Send a message to a Lambda function
2. Choose the Lambda function you'll create in the next step
3. Click Add action

Step 3: Secure Your Credentials

Using AWS Secrets Manager is the recommended approach for securely storing ServiceNow credentials.

3.1 Store the Credentials in AWS Secrets Manager

1. Go to the AWS Secrets Manager Console: [AWS Secrets Manager](https://console.aws.amazon.com/secretsmanager/)
2. Click Store a new secret
3. Select Other type of secret
4. Enter your ServiceNow credentials as key-value pairs:

    - servicenow_url: https://your-instance.service-now.com/api/now/table/incident
    - username: your_username
    - password: your_password
5. Click Next and provide a name for the secret (e.g., ServiceNowCredentials)
6. Complete the setup and save the secret

3.2 Attach Permissions to the Lambda Role

1. Go to the IAM Console: [AWS IAM Console](https://console.aws.amazon.com/iam/)

2. Find the execution role associated with your Lambda function
3. Attach the SecretsManagerReadWrite policy to the role:

    - Click Add permissions > Attach policies
    - Search for SecretsManagerReadWrite and attach it

3.3 Update the Lambda Function to Fetch the Secret

Replace hardcoded credentials in your Lambda function with code to retrieve the secret from AWS Secrets Manager:

```
import json
import urllib.request
import base64
import boto3

def get_secret():
    secret_name = "ServiceNowCredentials"  # Replace with your secret name
    region_name = "us-east-2"  # Replace with your AWS region

    # Create a Secrets Manager client
    client = boto3.client("secretsmanager", region_name=region_name)

    # Retrieve the secret
    response = client.get_secret_value(SecretId=secret_name)
    secret = json.loads(response["SecretString"])
    return secret

def lambda_handler(event, context):
    # Fetch credentials from Secrets Manager
    secret = get_secret()
    servicenow_url = secret["servicenow_url"]
    username = secret["username"]
    password = secret["password"]

    # Encode credentials for Basic Authentication
    credentials = f"{username}:{password}"
    encoded_credentials = base64.b64encode(credentials.encode()).decode()

    # Parse the IoT message (directly from the event object)
    sensor_id = event.get('sensor_id', 'unknown')
    temperature = event.get('temperature', 'unknown')
    sensor_type = event.get('sensors', 'unknown')

    # Create an incident in ServiceNow
    headers = {
        "Content-Type": "application/json",
        "Accept": "application/json",
        "Authorization": f"Basic {encoded_credentials}"
    }
    data = {
        "short_description": f"Alert from {sensor_type} sensor {sensor_id}",
        "description": f"Sensor {sensor_id} reported a temperature of
{temperature}°C.",
        "urgency": "2",  # Medium urgency
        "impact": "2"    # Medium impact
    }

    # Make the HTTP POST request to ServiceNow
    try:
        req = urllib.request.Request(servicenow_url,
data=json.dumps(data).encode(), headers=headers)
        response = urllib.request.urlopen(req)
        print(f"ServiceNow Response: {response.status},
{response.read().decode()}")
    except urllib.error.HTTPError as e:
        print(f"HTTPError: {e.code}, {e.reason}")
    except urllib.error.URLError as e:
        print(f"URLError: {e.reason}")

    return {
        "statusCode": 200,
        "body": "Alerts processed successfully"
    }
```

Click Deploy to save your Lambda function.

Important Note: Make sure to increase your Lambda function timeout to at least 15 seconds to allow sufficient time for the function to execute. The default 3-second timeout is often insufficient for this integration.

Step 4: Test the Integration

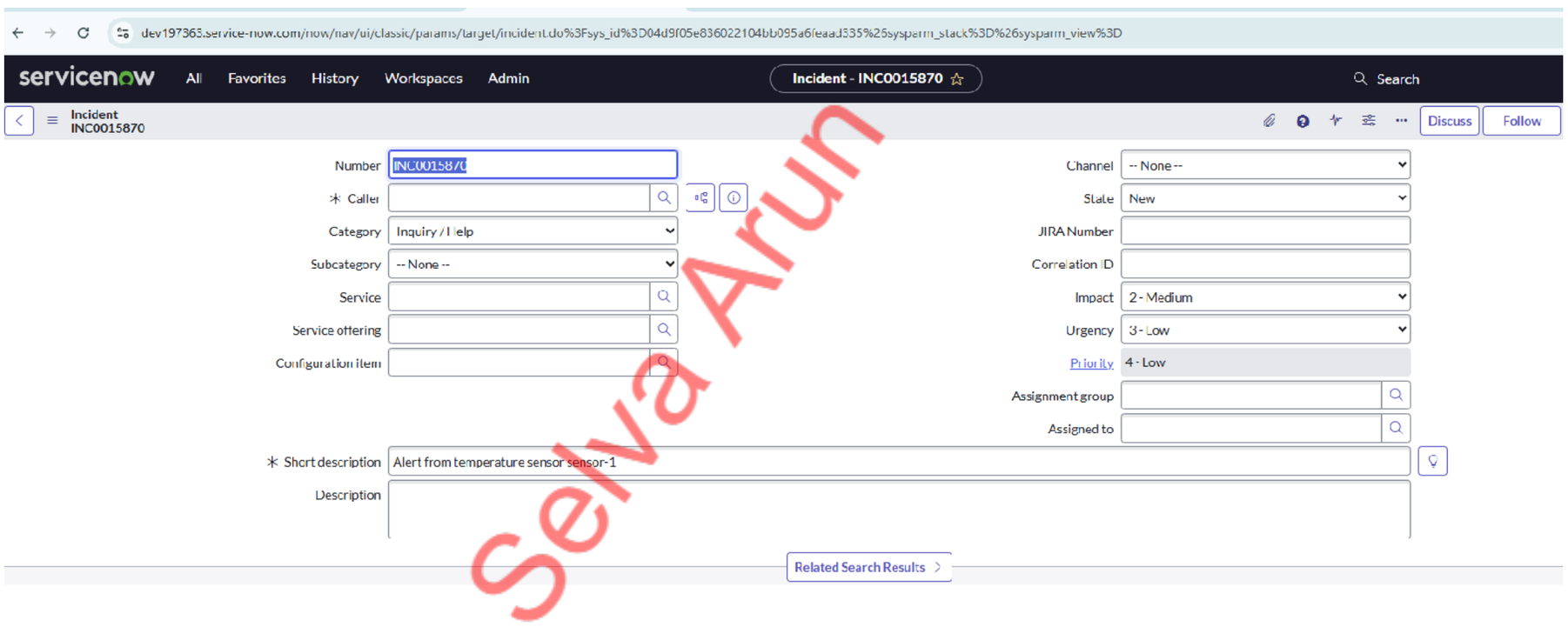
4.1 Publish a Test Message

Open Test > MQTT test client in AWS IoT Core and publish this message to the topic:

```
{
  "sensor_id": "sensor-1",
  "temperature": 75,
  "sensors": "temperature"
}
```

4.2 Verify Results

- Check CloudWatch Logs for Lambda execution details
- Log in to ServiceNow and verify incident creation



Results and Benefits

After successful integration, your system will automatically create incidents in ServiceNow when IoT sensors detect anomalies. Here's an example of what you'll see:

- **Incident Number:** INC0015870
- **Short Description:** Alert from temperature sensor sensor-1
- **Priority:** 4

Benefits:

- **Reduced Response Time:** Automatically detect and respond to potential issues before they impact business operations.
- **Minimized Human Error:** Eliminate manual incident reporting and ensure consistent documentation.
- **24/7 Monitoring:** Continuous monitoring without constant human supervision.
- **Standardized Documentation:** Consistent incident documentation with standardized formats.

Next Steps and Enhancements

This integration serves as a foundation that you can extend in several ways:

- Support multiple sensor types beyond temperature (humidity, power, motion, etc.)
- Add notifications using Amazon SNS to alert teams via email or SMS
- Store IoT data in Amazon DynamoDB or S3 for historical analysis and trend identification
- Automate the process of raising an incident via Flow Designer from ServiceNow (coming in Part 2 of this series)

Conclusion

This integration demonstrates the power of combining AWS IoT Core, Lambda, and ServiceNow for automated incident response. By implementing this solution, your organization can shift from reactive to proactive IT management, reducing downtime and improving operational efficiency. The beauty of this approach is its scalability—start with a single sensor type and expand as your needs grow. The same principles can be applied to various monitoring scenarios across your infrastructure.

I hope you found this guide helpful! Feel free to reach out with questions or share your own implementation experiences in the comments.

This article is Part 1 of a series on AWS IoT Core and ServiceNow integration. Stay tuned for Part 2, where I'll demonstrate building a Flow Designer solution for automating this process end-to-end.